

Working across two sites without moving the data: the Chalmers–Volvo Cars protocol

Prepared 2026-09-03 by Jonas Bårgman, Chalmers University of Technology, with drafting assistance from Claude (Anthropic). Status: DRAFT, for review and sign-off by Volvo Cars. The machine-readable form of the rules is the file `transfer/transfer_policy.yaml`, version 1, which the tooling enforces at both sites and which travels inside every transfer. This document explains those rules and the process around them. Where the two disagree, the policy file is what is enforced and this document is what needs correcting.

1 The situation

A method for measuring drivers' comfort-zone boundaries has been developed at Chalmers on video-based response studies. The next test is on naturalistic driving data. That data is held by Volvo Cars (VCC), cannot leave VCC, and the two organizations will not share a code repository or a drive. Files will move by e-mail. Both sides work with LLM assistants, which need the process written down precisely enough to follow it without judgment calls.

The purpose of this document is to agree, before any data is touched, on four things: what may cross between the sites, what never crosses, who checks each transfer, and how the record is kept so that any result can later be traced to the code and the settings that produced it.

2 Sites and roles

role	who	what they do
Home site (CTH)	Chalmers, Vehicle Safety division; lead Jonas Bårgman	owns the model, the analysis code and the documents; develops against synthetic data; receives aggregate results
Data site (VCC)	Volvo Cars; lead and steward to be named	holds the data; writes the adapter that produces the data interface; runs the shared code; exports aggregate results and code fixes after review
Data steward (VCC)	one named person at VCC	reviews and signs every transfer that leaves VCC; can approve recorded exceptions
LLM assistants	Claude at CTH; VCC's own assistant at VCC	work under <code>transfer/SITE_LLM_BRIEF.md</code> ; never move anything by hand

Each site keeps its own git repository. Nothing in the process requires the two to be the same repository or to have a common history; the transfers are the shared timeline.

3 Two layers and one interface

Every file at either site belongs to one of two layers.

The **shared layer** is what may cross: the model and analysis code, the tests, the documents, the transfer tooling itself, synthetic fixtures, and, from VCC, aggregate results. The **site layer** is what never crosses: raw data, any per-event or per-trip derived data, the scripts that read, clean and reshape the data, data paths, credentials, logs and notebooks. At VCC the site layer lives under `site/`; the policy file also names patterns (`**/ingest/**`, `**/data/**`, `*.parquet`, `*.log`, `*.ipynb` and others) that are refused wherever they sit.

The two layers meet at one **data interface**, `transfer/interface_schema.yaml`: one CSV per event with a fixed set of kinematic columns, one metadata table with one row per event, and an optional per-driver table with banded attributes only. Identifiers are opaque pseudonyms generated at VCC; the mapping to anything real stays at VCC. The interface carries no timestamps, no positions, no vehicle identifiers and no free text from logs. The shared code reads only this shape. VCC writes an adapter, in its site layer, that produces it; Chalmers develops against a synthetic fixture of the same shape (`transfer/make_synthetic_fixture.py`), so every shared script runs end to end at both sites before it meets data. A validator (`transfer/validate_interface.py`) checks an interface directory against the schema without printing a row of it.

4 What crosses, and what never does

category	CTH to VCC	VCC to CTH
model and analysis code, tests, tooling	yes	yes (fixes and additions)
documents (markdown)	yes	yes
synthetic fixtures	yes	no need
aggregate results: population summaries, per-cell tables with counts, held-out scores, fitted parameters with intervals, figures of aggregates, run records	not applicable	yes, after steward review
per-driver values (one number per pseudonymous driver)	not applicable	only as a recorded, steward-approved exception
per-event or per-trip rows, individual traces, excerpts of data	never	never
the adapter, any ingestion or preprocessing code, data paths, credentials	never	never
the pseudonym mapping	never	never
logs, notebooks, spreadsheets, binary data files	never	never

The aggregation rule. A row of a results table is an aggregate over at least a minimum number of drivers and of events, set in the policy as `min_n` (draft value 5), and no row identifies a trip, a drive, a vehicle, a day or a place. Tables carry a count column so the rule can be checked mechanically; a table without one is flagged for the steward. Figures follow the same rule as tables.

5 The transfer procedure

A transfer is a **bundle**: a zip archive holding the changed shared-layer files, a manifest with a cryptographic hash of every file and of the policy in force, and a review sheet. The tool `transfer/bundle.py` makes, checks and applies bundles; it refuses anything the policy refuses. Bundles are named `<site>-<date>-<sequence>`, for example `VCC-2026-09-10-001`.

- Make.** At the sending site: `bundle.py make --site VCC --to CTH --purpose "..."`. The tool works out what the other site does not yet have (§5a), keeps only files the policy allows, runs the content checks on each, and writes the zip, the manifest and the review sheet. If any check fails, no bundle is written. The first bundle from CTH carries the whole shared layer (`--all`).
- Check.** `bundle.py check <zip>` at either site re-verifies hashes and re-runs every policy check. Run it before sending and again on receipt.
- Steward review** (every bundle leaving VCC). The steward reads the review sheet, which lists each file with its kind, size, hash and check result and every warning, ticks the four items on it, signs it, and only then releases the bundle. An exception (for example, per-driver fitted levels) is recorded in the bundle itself with `--override "<who approved what, when>"` and appears on the sheet; the steward's signature covers it.
- Send.** The zip and the signed sheet go by e-mail.
- Apply.** At the receiving site: `bundle.py apply <zip> --commit`. The tool checks again, detects files edited locally since the sender's base and stops on a conflict (merge by hand, then `--force`), writes the files, records the manifest and the sheet, appends to the transfer log and commits with the bundle id in the message.
- Record.** Both sites keep `transfer/TRANSFER_LOG.md` and `transfer/manifests/`. If the two logs ever disagree, the manifests inside the bundles settle it.
- Revoke, if a bundle is not released.** A bundle the steward rejects, or that is never e-mailed, is marked with `bundle.py revoke <id> --reason "..."` so that its files are sent again next time (§5a). The tool prints this reminder after every make.

5a Only what has changed is sent, and how that is established

After the first shipment, a bundle carries only the files the other site does not already hold. This cannot be established from version control: the two sites share no history, and each site's commits describe only itself. It is established by content, using the SHA-256 hashes the manifests already carry.

Every manifest records the hash of each file the bundle carried, and also of the sender's entire exportable shared layer at that moment. To prepare a new bundle, the tool replays the manifests of all bundles exchanged with that partner — sent and received alike, since either direction leaves both sites holding the same content for the files it carried — and compares the result against the working tree. Files whose hash differs are included; files genuinely

deleted from disk are listed as deletions; everything else is left out, and the review sheet states how many files were unchanged and therefore not re-sent. `bundle.py peers` shows the current picture at any time.

Two safeguards attach to this. A bundle that is made but never released would otherwise leave the tool believing the partner had received it, which is what revocation (step 7) exists for. And because each manifest carries the sender's whole tree, the receiving site can compare itself against it: `apply` reports any file the sender holds that is missing or different locally, so a file that should have arrived but did not is visible rather than silent.

6 Traceability of results

Every number that reaches a document at Chalmers comes from a committed script with a tracked output; that standing rule extends across the sites. A result table exported from VCC carries a **run record**: the script's name and commit id at VCC, the parameter values, the date, and the validator's report on the interface directory it was run on. It never carries the data path. The chain from a quoted number back to the code that produced it is therefore: the document at CTH, the bundle id, the manifest's file hash, the run record, the commit at VCC.

7 Rules for the LLM assistants

The assistants at both sites work under `transfer/SITE_LLM_BRIEF.md`. The rules that matter most, in one place:

- Nothing moves from the site layer to the shared layer: no path, no row, no screenshot, no real value pasted as an example. Synthetic fixtures are the examples.
- No data content in chat, documents, commit messages or questions; data is described by shape and counts.
- Results leave only through scripts writing into `results/aggregate/`, with run records.
- `bundle.py scan` runs before every make; a refused file is fixed or taken to the steward, never waved through by changing a rule.
- Received bundles are applied with the tool, never by hand.
- Notes and questions to the other site go into that site's own append-only notes file (`docs/notes_from_CTH.md`, `docs/notes_from_VCC.md`); neither site edits the other's.
- When in doubt whether something may leave, it may not.

8 What Volvo Cars is asked to decide

- 1 The minimum count `min_n` for aggregates (draft: 5 drivers and 5 events per row).
- 2 Whether per-driver fitted values (one number per pseudonym, no attributes) may ever leave as a recorded exception, or never.
- 3 Whether figures may show individual points, or aggregates only (draft: aggregates only).
- 4 The names of the VCC lead and the data steward.
- 5 Additional patterns the export scan should refuse (the names of VCC's data shares and internal hostnames, for the `site_path` rule).
- 6 Whether VCC wants to review the interface schema before adapter work starts (recommended).
- 7 Retention: how long Chalmers keeps received bundles and whether results may be quoted in publications under an agreed review step.

9 Change control and sign-off

The policy file carries a version number and a status line. Changing any rule produces a new version, which both sides sign again; the tool records the policy's hash in every manifest, so any bundle can be checked against the version that was in force when it was made. The transfer tooling itself is part of the shared layer and moves in bundles like any other code, so both sites always run the same version.

	name	date	signature
Volvo Cars, data steward			
Volvo Cars, project lead			
Chalmers, project lead	Jonas Bärman		

Policy version signed: 1 (`transfer/transfer_policy.yaml`, status DRAFT until signed).

Appendix A: the policy file, in brief

transfer/transfer_policy.yaml has five parts. sites names each site and its role. transfer sets the channel and the size limits (3 MB per file, 25 MB per bundle, the usual e-mail ceiling) and which file types are text (scanned) and which binaries are allowed (images and PDFs only; every other binary is refused). roles.home and roles.data each give an allow list (a file must match one pattern to be bundled), a never list (a match refuses the file whatever else it matches), forbidden_patterns (regular expressions whose match refuses the bundle: drive and share paths, Unix data paths, vehicle identification numbers, coordinate pairs in the Nordic range, wall-clock timestamps) and warn_patterns (flagged for the steward). For the data site it also sets the table rules: which paths count as aggregate tables, which column names are forbidden, which columns are counts, min_n, the row limit, and whether per-person rows are allowed. sign_off holds the signatures.

Appendix B: commands

```
python transfer/bundle.py make --site CTH --to VCC --purpose "." --all      # first bundle only
python transfer/bundle.py make --site VCC --to CTH --purpose "."           # every later bundle
python transfer/bundle.py check transfer/inbox/<bundle>.zip
python transfer/bundle.py apply transfer/inbox/<bundle>.zip --commit
python transfer/bundle.py scan --site VCC
python transfer/bundle.py peers --site VCC
python transfer/bundle.py revoke <bundle-id> --reason "not released"
python transfer/bundle.py drift transfer/inbox/<bundle>.zip
python transfer/bundle.py selftest
python transfer/validate_interface.py <interface directory>
python transfer/make_synthetic_fixture.py
python tests/test_transfer.py
```